

机器学习 ——强化学习

Kai Chen

kaichen6@csu.edu.cn

1st October 2025



中南大学
CENTRAL SOUTH UNIVERSITY

数学与统计学院
SCHOOL OF MATHEMATICS AND STATISTICS

1 学习介绍与分类

2 强化学习

3 寻找最优策略

- 规划算法
- 学习算法

什么是学习？

学习发生在 **智能体 (agent)** 与 **世界 (world)** 的交互过程中，其思想是：

- 智能体接收到的感知 (Perception) 不仅用于理解 / 解释 / 预测

在传统的机器学习任务中，如分类或回归，模型通常被训练来理解数据、解释数据中的模式或预测未来的数据点

- 感知信息还用于指导行动，即智能体根据感知来决定采取哪些动作

- 监督学习

- 给出或可以观察到函数要学习的样本（输入，输出）对；
- 可以想象成有一位“老师”；
- 训练数据： (X, Y) （特征，标签）；
- 预测 Y ，最小化某种损失；
- 回归、分类。

- 无监督学习

- 训练数据： X （仅特征）；
- 在高维 X 空间中寻找“相似”点。

监督学习示例

- 根据经济数据预测 6 个月后某只股票的价格 (回归);
- 预测一位因心脏病住院的患者是否会再次发作;
 - 基于人口统计、饮食和临床测量 (逻辑回归)。
- 从数字化图像 (像素) 中识别手写邮政编码中的数字 (分类)。

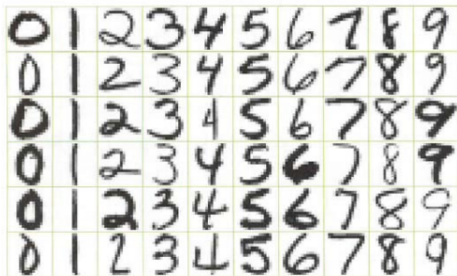


FIGURE 1.2. Examples of handwritten digits from U.S. postal envelopes.

无监督学习示例

从 DNA 微阵列数据中，确定哪些基因在表达谱方面最“相似”（聚类）。



强化学习

- 智能体对环境采取行动，收到对其行动的某种评价 (**reinforcement**)，但并未被告知应采取哪一行动才能实现目标；
- 训练数据： (S, A, R) (**状态-动作-奖励**)；
- 开发**最优策略** (**policy**) (决策规则序列)，使学习者长期奖励最大化；
- 如机器人、博弈程序。

Outline

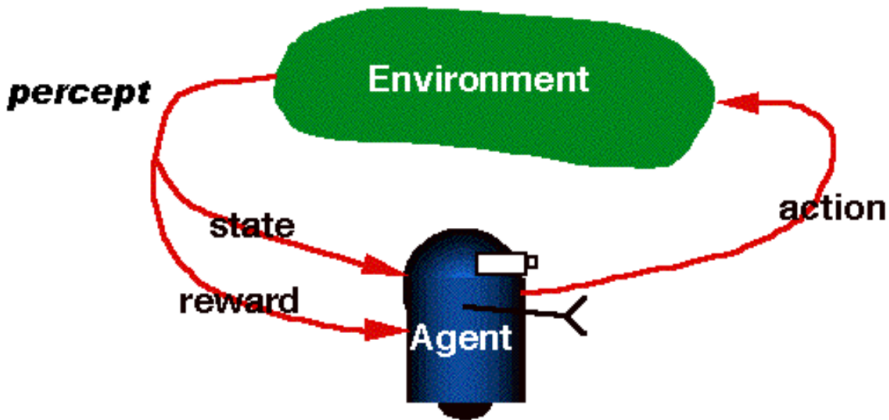
1 学习介绍与分类

2 强化学习

3 寻找最优策略

- 规划算法
- 学习算法

RL 是从交互中学习



强化学习示例

- 机器人如何行为才能优化其“表现”？(机器人学)
- 如何自动化直升机运动？(控制理论)
- 如何制作好的国际象棋程序？(人工智能)



机器人学

- 动作空间：上、下、左、右；
- 向上移动成功率 80%，左右各 10%；
- 在 $[4,3]$ 奖励 +1， $[4,2]$ 奖励-1，每步奖励-0.04；

			+1
			-1
START			

问：如何策略才能最大化奖励？若动作非确定性又如何？

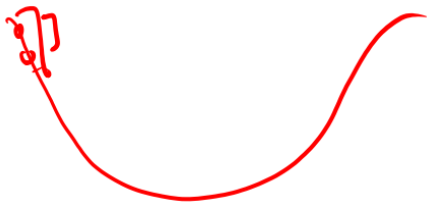
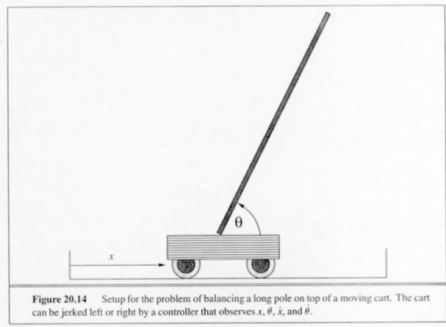
杆平衡任务

任务：

- 左右移动小车以保持杆子平衡

状态表示

- 小车的位置和速度
- 杆子的角度和角速度



强化学习历史

- 源于动物学习心理学;
- 另一独立线索是**最优控制问题** (optimal control)及其使用**动态规划**的解法 (Bellman);
- **时间差分学习** (temporal difference learning)(on-line method) 思想, 例如博弈程序 (Samuel);
- 重大突破是**Q-learning**的发现 (Watkins,1989)。

强化学习有何特别？

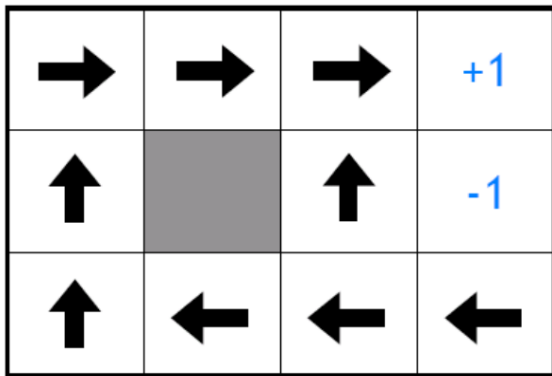
- RL 学习如何**将状态映射到动作**，以最大化数值奖励；
- 与其他学习不同，它是一个多阶段决策过程（通常是**马尔可夫的**）；
- RL 智能体必须通过**试错学习**（非完全监督，但交互式）；
- 动作不仅影响即时奖励，还影响后续奖励（**延迟奖励**）。

强化学习要素

- **策略 (policy):**
从状态空间到动作空间的映射;
- **奖励函数 (reward function):**
将每个状态 (或状态-动作对) 映射到实数;
- **价值函数 (value function):**
状态 (或状态-动作对) 的值为从该状态开始的总期望奖励。

策略

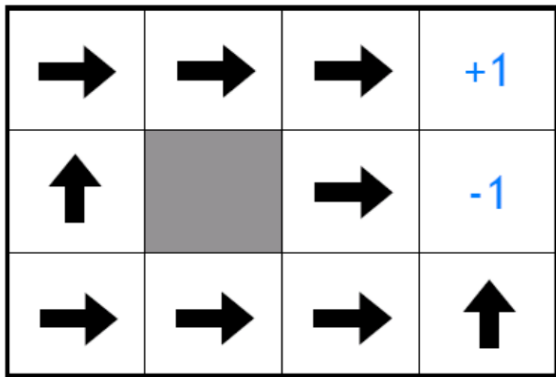
策略是智能体在给定状态下应采取的动作的映射。



如果智能体处于中心状态，它可以选择向上、向下、向左或向右移动。

每步奖励: -2

如果采取每个动作后获得的奖励都为-2。这意味着无论智能体采取哪个动作, 都会受到惩罚。



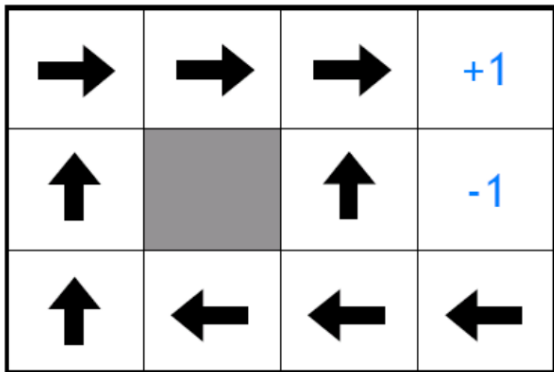
每步奖励：-0.1

如果采取每个动作后获得的奖励为-0.1。这表示智能体受到的惩罚较轻。

→	→	→	+1
↑		↑	-1
↑	→	↑	←

每步奖励: -0.04

如果采取每个动作后获得的奖励为 -0.04 。这表示智能体受到的惩罚最轻。



精确目标

- 找到一个**最大化价值函数的策略**。
 - transitions 和 rewards 通常不可用
- 在不同情况下实现这一目标有不同的方法。
- **值迭代 (Value iteration)**和**策略迭代 (Policy iteration)**是解决这个问题的两种更经典的方法。但本质上，两者都是**动态规划**。
- **Q 学习**是解决这个问题的一种较新的方法。本质上，它是一种**时序差分方法**。

马尔可夫决策过程

马尔可夫决策过程 (Markov decision process, MDP) 是一个五元组: $(S, A, P_{sa}, \gamma, R)$

- S 是状态集合。
(例如, 在自主直升机飞行中, S 可能是直升机所有可能的位置和方向的集合。)
- A 是动作集合。
(例如, 你可以推动直升机控制杆的所有可能方向的集合。)
- P_{sa} 是状态转移概率。
对于每个状态 $s \in S$ 和动作 $a \in A$, P_{sa} 是状态空间上的一个分布。 P_{sa} 给出了如果我们在状态 s 下采取动作 a , 我们将转移到哪些状态的分布。
- $\gamma \in [0, 1)$ 被称为折扣因子 (discount factor), 是一个常数。
- $R: S \times A \mapsto \mathbb{R}$ 是奖励函数。
(奖励有时也仅作为状态 S 的函数来写, 在这种情况下, 我们将有 $R: S \mapsto \mathbb{R}$ 。)

马尔可夫决策过程

- 确定性环境的 MDP

状态转移函数和奖励函数都是确定性

例如在下围棋时，在同样一个局势中下相同的子，得到的新局势和奖励是确定的

- 非确定性环境的 MDP

无法确定状态转移和奖励

我们从一个状态 s_0 开始，并选择一个动作 $a_0 \in A$ 。

- 由于我们的选择，MDP 的状态随机转移到某个后继状态 s_1 ，根据 $s_1 \sim P_{s_0 a_0}$ 抽取。
- 然后，我们选择另一个动作 a_1 。
- ...

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} \dots$$

- 在访问状态序列 $s_0, s_1, \dots$ 并采取动作 $a_0, a_1, \dots$ 时，我们的总收益由以下公式给出：

$$R(s_0, a_0) + \gamma R(s_1, a_1) + \gamma^2 R(s_2, a_2) + \dots$$

- 或者，当我们仅将奖励作为状态的函数来写时，这变为：

$$R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$

- 在我们的大部分开发中，我们将使用更简单的状态奖励 $R(s)$ ，尽管推广到状态-动作奖励 $R(s; a)$ 并没有特别的困难。
- 在强化学习中，我们的目标是随着时间的推移选择动作，以便最大化总收益的期望值：

$$\mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots]$$

- 策略是从状态到动作的映射函数 $\pi : S \mapsto A$ 。
- 当我们在状态 s 时，执行某个策略意味着我们采取动作 $a = \pi(s)$ 。
- 我们还定义了策略 π 的价值函数为

$$V^\pi(s) = \mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots \mid s_0 = s, \pi]$$

- $V^\pi(s)$ 简单地表示从状态 s 开始，并根据 π 采取动作时，折扣奖励的期望总和。

价值函数

给定一个固定策略 π , 其价值函数 V^π 满足贝尔曼方程 (Bellman equations):

$$V^\pi(s) = R(s) + \gamma \sum_{s' \in S} P_{s\pi(s)}(s') V^\pi(s')$$

- 即时奖励
- 未来折扣奖励的期望和

贝尔曼方程可以用来有效求解 V^π

The Grid world

$M = 0.8$ in direction you want to go
0.2 in perpendicular $\begin{cases} 0.1 \text{ left} \\ 0.1 \text{ right} \end{cases}$

策略：从状态到动作的映射

3	→	→	→	+1
2	↑		↑	-1
1	↑	←	←	←
	1	2	3	4

3	0.812	0.868	0.912	+1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388
	1	2	3	4

环境

- **可观测 (可访问)**：感知器能够识别状态
智能体不需要进行任何额外的推断或猜测就能知道当前状态
- **部分可观测**
智能体需要根据不完全的信息来做出决策

在实际应用中，许多环境都是部分可观测的

- **马尔可夫性质**：状态转移概率仅依赖于当前状态，而不依赖于达到该状态的路径。
- **马尔可夫决策问题 (MDP)**。
- **部分可观测马尔可夫决策问题 (POMDP)**：感知信息不足以确定状态转移概率。

最优价值函数

我们定义最优价值函数为

$$V^*(s) = \max_{\pi} V^{\pi}(s),$$

换句话说，这是使用任意策略所能获得的折扣奖励期望总和的最佳可能值。
对于最优价值函数，也有对应的贝尔曼方程：

$$V^*(s) = R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') V^*(s').$$

最优策略

我们还定义一个策略 $\pi^*: S \rightarrow A$ 如下:

$$\pi^*(s) = \arg \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') V^*(s')$$

事实:

$$V^*(s) = V^{\pi^*}(s) \geq V^{\pi}(s), \quad \forall \pi.$$

- 策略 π^* 具有一个有趣的性质: 它对所有状态 s 都是最优策略。
- 并不存在 “若从某个状态 s 开始, 则存在仅对该状态最优的策略; 若从另一个状态 s' 开始, 则又存在仅对 s' 最优的另一策略” 的情况。
- 同一个策略 π^* 对所有状态 s 都能达到式中的最大值。
- 这意味着无论 MDP 的初始状态为何, 我们都可以使用同一个策略 π^* 。

Outline

- 1 学习介绍与分类
- 2 强化学习
- 3 寻找最优策略
 - 规划算法
 - 学习算法

学习的基本设定

训练数据： n 条有限视界轨迹

$$\{s_0, a_0, r_0, \dots, s_T, a_T, r_T, s_{T+1}\}$$

确定性或随机性策略： 决策规则序列

$$\{\pi_0, \pi_1, \dots, \pi_T\}$$

每个策略 π 将可观测的历史（即之前的状态和动作序列）映射到当前时刻的动作空间。

寻找最优策略

- 规划算法

假设环境模型是已知的

所有的 $s, s' \in S$ 和 $a \in A$, 转移概率 $\Pr[s'|s, a]$ 和期望奖励 $E[r(s, a)]$ 都是已知的

- 学习算法

马尔可夫决策过程 (MDP) 的环境模型, 也就是转移概率和奖励概率, 是未知的

- 无模型方法, 包括直接学习一个动作策略
- 模型基方法, 包括首先学习环境模型, 然后利用该模型来学习策略

Outline

- 1 学习介绍与分类
- 2 强化学习
- 3 寻找最优策略
 - 规划算法
 - 学习算法

算法 1: 价值迭代

- 仅考虑具有有限状态和动作空间的马尔可夫决策过程 (MDPs) ($|S| < \infty, |A| < \infty$)。

- 价值迭代算法:**

- 对每个状态 s , 初始化 $V(s) \leftarrow 0$ 。
- 重复直到收敛 {
 - 对每个状态, 更新

$$V(s) := R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') V^*(s').$$

}

- 同步更新:** 所有的状态价值在每一次迭代中同时更新
- 异步更新:** 随机或按照某种顺序选择一部分状态进行更新
- 可以证明, 价值迭代将使 V 收敛到 V^* 。找到 V^* 后, 我们可以使用方程 $\pi^*(s) = \arg \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') V^*(s')$ 找到最优策略。

算法 2: 策略迭代

- 策略迭代算法:

- 1 随机初始化 π 。

- 2 重复直到收敛 {

- 设 $V := V^\pi$ 。

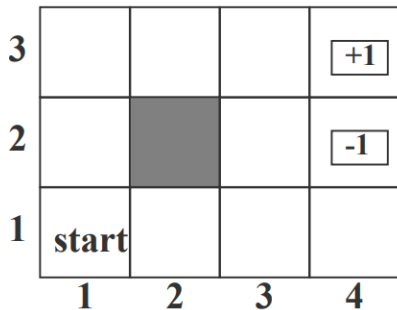
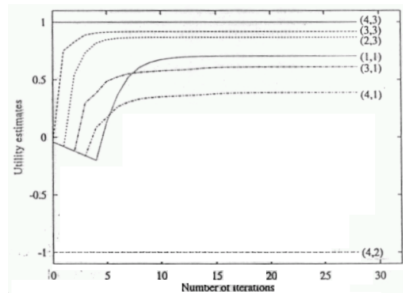
- 对于每个状态 s , 令 $\pi(s) := \arg \max_{a \in A} \sum_{s' \in S} P_{sa}(s') V^*(s')$ 。

- }

- 内循环反复计算当前策略的价值函数, 然后使用当前价值函数更新策略。
- 贪婪更新: 在每个状态下选择当前最优的动作来逐步改进策略
- 在有限次迭代后, V 将收敛到 V^* , π 将收敛到 π^* 。

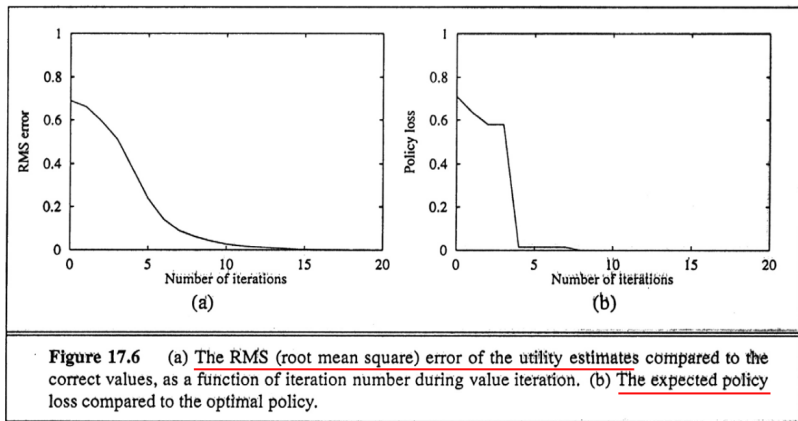
收敛性

在应用价值迭代到我们的示例中的 4×3 世界时，每个迭代步骤中选定状态的效用值



当 $t \rightarrow \infty$ 时，即使更新是异步进行的，并且每一步随机选择 i ，价值迭代也会收敛到精确的效用值 U

何时停止值迭代?



图示：RMS 误差与期望策略损失随迭代次数下降。
什么时候停止值迭代

Outline

- 1 学习介绍与分类
- 2 强化学习
- 3 寻找最优策略
 - 规划算法
 - 学习算法

策略的动作价值函数: $Q_\pi : S \times A \rightarrow \mathbb{R}$, 任意状态 $s \in S$ 下采取动作 $a \in A$ 后, 再执行策略 π 能够取得的回报的期望,

$$\begin{aligned} Q_\pi(s, a) &= R(s, a) + \mathbb{E}_{s_t, a_t} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right] \\ &= \mathbb{E}_{s_1} [R(s, a) + \gamma V_\pi(s_1) \mid s_0 = s, a_0 = a] \end{aligned}$$

贝尔曼最优方程

$$\begin{aligned} V^*(s) &= \max_{a \in A} Q^*(s, a), \\ Q^*(s, a) &= R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s') \end{aligned}$$

时序差分 (Temporal Difference, TD) 方法 + Q 学习 (Q-learning)

- **时序差分方法**: 利用当前状态和下一个状态的信息来预测未来奖励, 而不是等待整个序列结束才更新价值估计
- **TD-Q 学习** 不需要环境的模型 (即状态转移概率和奖励函数), 它直接从与环境的交互中学习
- 选择动作的规则

$$a = \arg \max_a Q(s, a)$$

算法 3: Q 学习

对于每对 (s, a) , 初始化 $Q(s, a)$

观察当前状态 s

永远循环

- 选择一个动作 a (可选地使用 ϵ -探索) 并执行它

$$a = \arg \max_a Q(s, a)$$

- 接收即时奖励 r 并观察新状态 s'
- 更新 $Q(s, a)$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- $s \leftarrow s'$

- 在利用（控制）和探索（识别）之间的权衡
- 极端情况：贪婪策略 vs. 随机行动（n 臂赌博机模型）
贪婪策略：在给定状态下总是选择当前估计最优的动作
随机行动：在给定状态下随机选择动作，不考虑当前估计的优劣
n 臂赌博机模型：一种简单的强化学习模型，其中有 n 个臂（动作），每个臂都有一个未知的奖励概率分布。目标是通过选择不同的臂来最大化累积奖励。

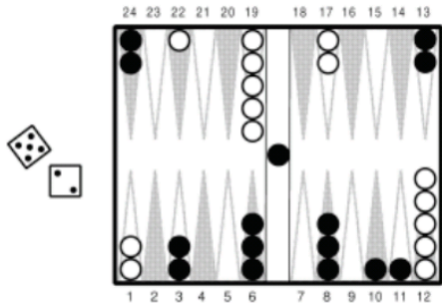
Q 学习收敛到最优 Q 值的条件：

- 每个状态被无限次访问（由于探索）
- 随着时间接近无穷大，动作选择变得贪婪
- 学习率决定了新信息对旧估计的影响程度。学习率 α 需要足够快地减小，但不能太快（如我们在时序差分学习中讨论的）

成功案例：TD-Gammon

TD Gammon (Tesauro, G., 1992)

- 一个玩西洋双陆棋的程序。
- 时序差分学习的应用。
- 基本的学习器是一个神经网络。
- 它通过与自身对弈并从结果中学习，自我训练到世界级水平。
- 更多信息：<http://www.research.ibm.com/massive/td1.html>



- 价值迭代和策略迭代都是解决马尔可夫决策过程 (MDP) 的标准算法，目前还没有普遍的共识来确定哪种算法更好。
- 对于小型 MDP，价值迭代通常非常快，并且在很少的迭代次数内就能收敛。然而，对于具有大型状态空间的 MDP，显式求解价值函数 V 将涉及到求解一个大型线性方程组，这可能会很困难。
- 在这些问题中，策略迭代可能更受偏好。在实践中，价值迭代似乎比策略迭代更常用。
- Q 学习是无模型的，并且探索时序差分。

学习的类型

- 监督学习

- 训练数据: (X, Y) (特征, 标签)
- 预测 Y , 最小化某些损失。
- 回归, 分类。

- 无监督学习

- 训练数据: X (仅特征)
- 在高维 X 空间中找到“相似”点。
- 聚类。

- 强化学习

- 训练数据: (S, A, R) (状态-动作-奖励)
- 为学习者开发一个最优策略 (决策规则序列), 以便最大化其长期奖励。
- 机器人技术, 棋盘游戏程序。

Feedback welcome!

The End